HackDNA – Secrets in Source 2

Challenge Overview

The application exposes sensitive information directly inside the client-side source code. Attackers commonly inspect HTML, JavaScript files, comments, hidden fields, API endpoints, and embedded configuration values to discover secrets unintentionally left by developers.

This challenge demonstrates how insecure source-code exposure can lead to information disclosure and privilege escalation.

Objective

Identify hidden credentials, tokens, or sensitive information exposed within the web application's source code.

Reconnaissance

The first step involves inspecting the application source.

Open the target page and view the source code:

```
CTRL + U
```

or

```
Right Click → View Page Source
```

Developers sometimes leave:

- Credentials
- API Keys
- Debug Comments
- Hidden Endpoints
- Developer Notes
- Backup URLs
- JWT Tokens
- Internal IP Addresses

inside the source.

Initial Findings

While reviewing the HTML source, suspicious comments and hidden values may appear.

Example indicators:

```
<!-- TODO: remove test credentials before production -->
```

```
<input type="hidden" value="admin:true">
```

```
const api_key = "dev-test-key-123";
```

Client-side JavaScript files are also critical.

JavaScript Enumeration

Inspect loaded JavaScript files using browser developer tools.

```
F12 → Sources
```

or inspect script references:

```
<script src="app.js"></script>
```

Download and review the scripts.

Useful patterns to search:

```
password
secret
key
token
admin
```

```
internal
backup
```

Exploitation

A sensitive value exposed in the source can often be reused for:

- Authentication bypass
- Admin access
- Hidden functionality
- API interaction
- Privilege escalation

Example:

```
const adminPassword = "SuperSecret123";
```

Using the discovered credential on the login page grants elevated access.

Root Cause

The vulnerability exists because sensitive information was embedded directly into client-side code.

Anything sent to the browser must be considered public.

Common developer mistakes include:

- Hardcoded credentials
- Debugging leftovers
- Exposed API secrets
- Development endpoints left enabled
- Insecure comments containing operational data

Security Impact

Source-code disclosure vulnerabilities can lead to:

- Unauthorized access
- Credential compromise

- Internal infrastructure exposure
- Account takeover
- API abuse
- Privilege escalation
- Full application compromise

Detection Techniques

Manual Inspection

- View page source
- Review JavaScript files
- Inspect comments
- Analyze hidden fields
- Review network requests

Automated Discovery

Using grep:

```
grep -Ri "password\|secret\|token\|apikey" .
```

Using browser DevTools:

```
F12 → Network → JS Files
```

Mitigation

Never Store Secrets Client-Side

Sensitive credentials should remain server-side.

Remove Debug Information

Strip comments and development notes before deployment.

Use Environment Variables

Secrets should be managed securely using:

- Environment variables
- Vault systems
- Secret managers

Conduct Secure Code Reviews

Implement:

- Static analysis
- CI/CD secret scanning
- Manual security audits

Apply Principle of Least Privilege

Even exposed tokens should have minimal permissions.

Example Secure Practice

Instead of:

```
const db_password = "root123";
```

Use server-side authentication logic and environment variables.

Real-World Relevance

Many real-world breaches originate from accidentally exposed secrets in:

- Public Git repositories
- Frontend JavaScript
- Mobile applications
- CI/CD pipelines
- Backup files

Common exposed secrets include:

- AWS Keys

- Firebase credentials
- JWT secrets
- SMTP credentials
- Database passwords
- OAuth tokens

Key Takeaway

If the browser can see it, an attacker can see it.

Client-side code should never contain sensitive operational secrets.

Vulnerability Classification

- CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
- CWE-798: Use of Hard-coded Credentials
- OWASP A05:2021 – Security Misconfiguration
- OWASP A02:2021 – Cryptographic Failures

Conclusion

Secrets exposed in source code represent a high-risk information disclosure vulnerability. Attackers routinely inspect frontend assets during reconnaissance, making exposed credentials one of the easiest attack vectors to exploit.

Secure development practices, automated secret scanning, and proper server-side secret management are essential to prevent this class of vulnerability.